Laboratorio 3: Patrón Arquitectónico Broker

Santiago Suaza Montalvo

13 de abril de 2026

Introducción

Este informe documenta la implementación exitosa del patrón Broker. Se demuestra el desacoplamiento entre un productor (FastAPI) y un consumidor (Python Worker) mediante el uso de RabbitMQ como intermediario y MongoDB para la persistencia de datos.

Component-and-Connector View

La arquitectura implementa el patrón **Broker** con cuatro componentes conectados mediante dos tipos de conectores: colas AMQP (asíncronas y durables) y una conexión de base de datos síncrona (`db_connector`).

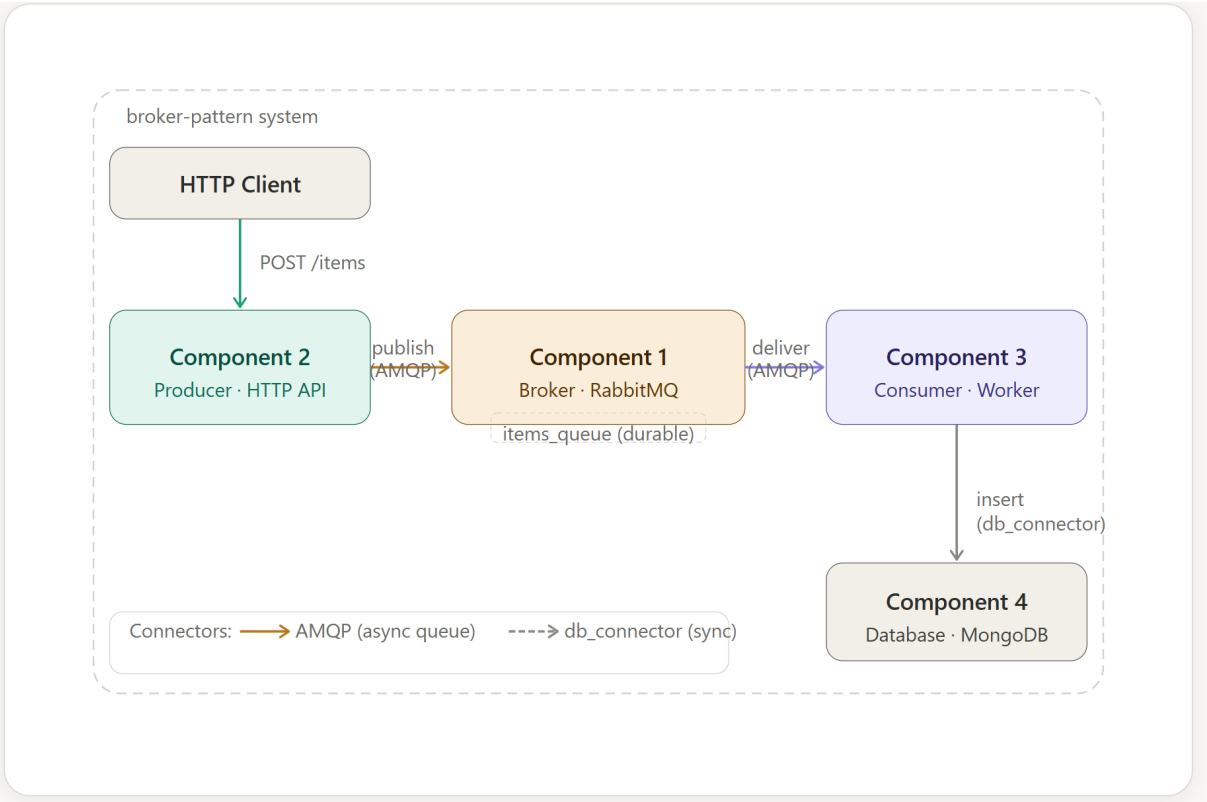


Figura 1: Component-and-Connector View — Patrón Broker

El **Component 1** (RabbitMQ) actúa como broker central: recibe mensajes del productor y los enruta al consumidor a través de la cola durable `items_queue`. El **Component 2** (Producer) expone una HTTP API; el **Component 3** (Consumer) es un worker que suscribe la cola y persiste los mensajes; el **Component 4** (MongoDB) almacena los datos procesados.

Descripción de la comunicación asíncrona

La comunicación entre el Producer y el Consumer es completamente asíncrona, mediada por la cola `items_queue` del broker.

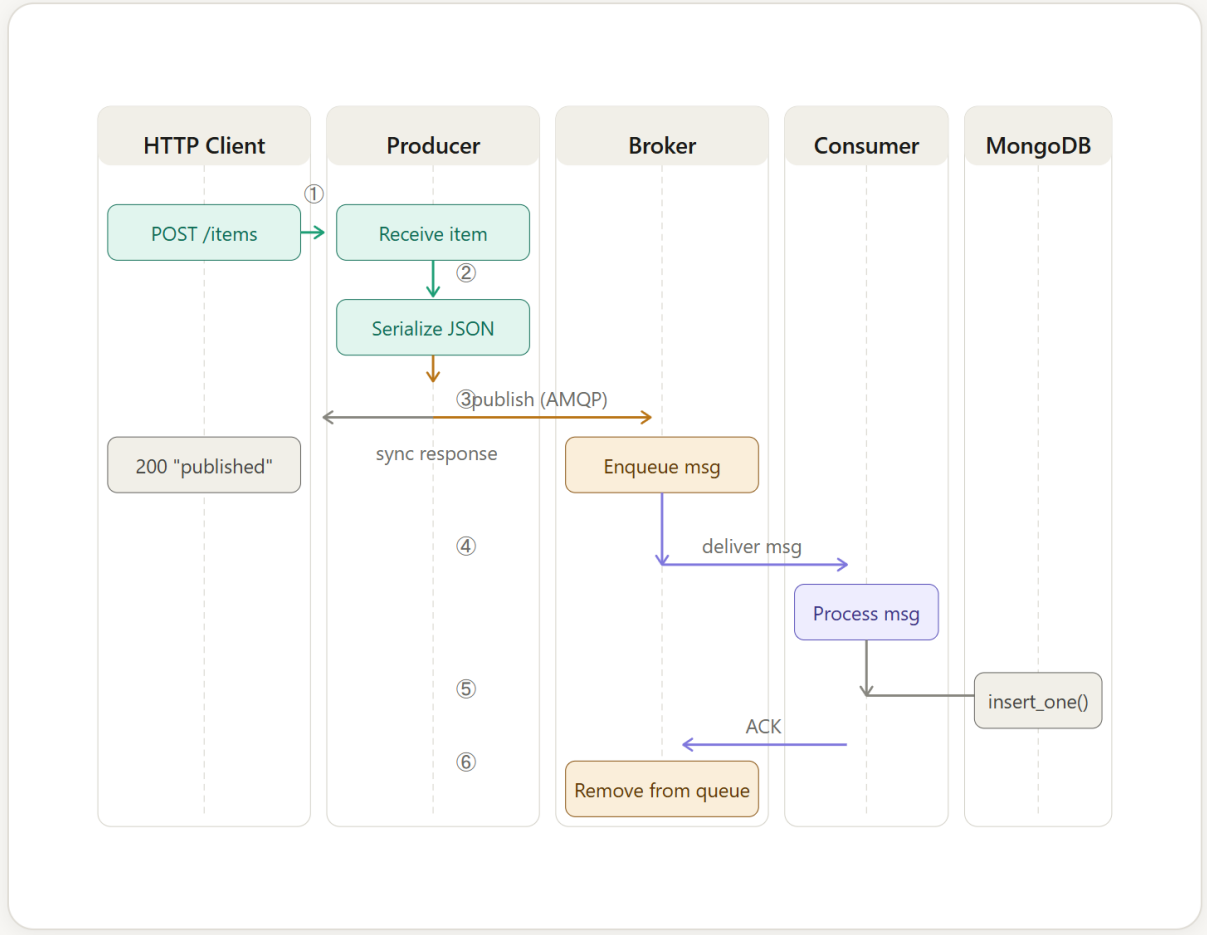


Figura 2: Flujo de comunicación asíncrona — Patrón Broker

1. **Fase síncrona:** El cliente HTTP envía `POST /items` al Producer. Este serializa el mensaje en JSON y lo publica al broker vía AMQP. En cuanto el broker confirma la recepción en la cola, el Producer retorna inmediatamente {`"status": "published"`} al cliente.

2. **Fase asíncrona:** De forma independiente, el broker entrega el mensaje al Consumer. Este lo procesa y lo persiste en MongoDB mediante `insert_one()`. Finalmente, el Consumer envía un **ACK** al broker.

Evidencias de Implementación

Petición del Productor (Curl)

```
ssuazam@clickandmuch:~$ curl -X POST http://localhost:8000/items \
> -H "Content-Type: application/json" \
-d '{"name": "SwArch", "description": "2025-II"}'
{"status":"published"}ssuazam@clickandmuch:~$
```

Figura 3: Ejecución del comando curl y respuesta del Productor.

Estado del Message Broker (RabbitMQ)

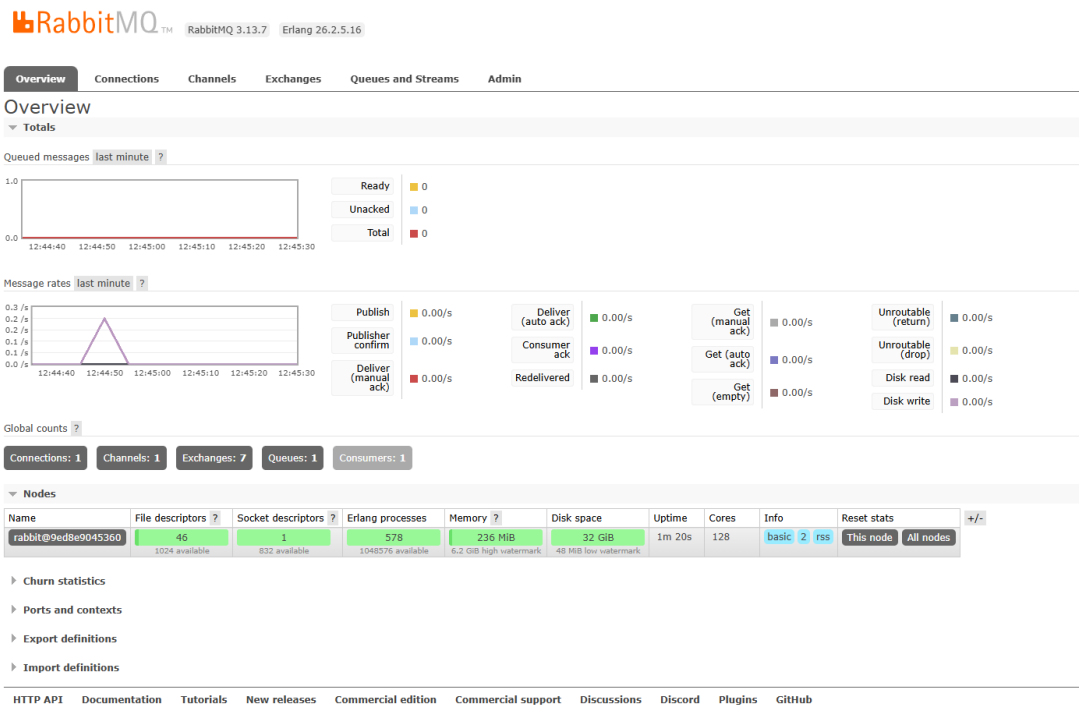


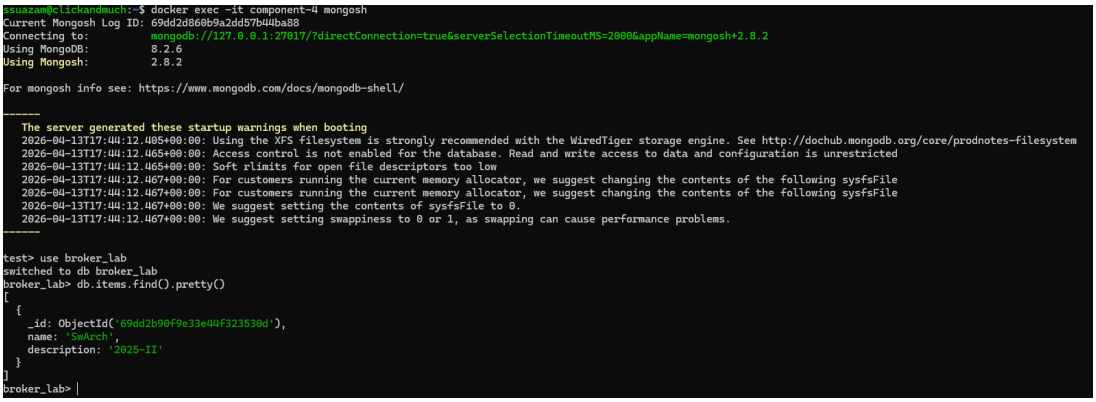
Figura 4: Estado de las colas y tasas de mensajes en el Broker.

Logs del Consumidor (Componente 3)

```
component-3  | [x] Item stored: {'name': 'SwArch', 'description': '2025-II', '_id': ObjectId('69dd2b90f9e33e44f323530d')}
```

Figura 5: Logs de salida del Componente 3 confirmando el almacenamiento.

Persistencia de Datos (MongoDB)



```
ssuazam@clickandmuch:~$ docker exec -it component-4 mongosh
Current Mongosh Log ID: 69dd2d860b9a2dd57b44ba88
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2
Using MongoDB:          8.2.6
Using Mongosh:          2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-04-13T17:44:12.405+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
   2026-04-13T17:44:12.465+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
   2026-04-13T17:44:12.465+00:00: Soft rlimits for open file descriptors too low
   2026-04-13T17:44:12.467+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
   2026-04-13T17:44:12.467+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
   2026-04-13T17:44:12.467+00:00: We suggest setting the contents of sysfsFile to 0.
   2026-04-13T17:44:12.467+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
------

test> use broker_lab
switched to db broker_lab
broker_lab> db.items.find().pretty()
[
  {
    _id: ObjectId('69dd2b90f9e33e44f323530d'),
    name: 'SwArch',
    description: '2025-II'
  }
]
broker_lab>
```

Figura 6: Consulta de registros en MongoDB mediante mongosh.

Conclusiones

Las evidencias recolectadas confirman que la arquitectura cumple con las propiedades del patrón Broker:

1. **Desacoplamiento:** El productor no conoce la existencia de la base de datos.
2. **Asincronía:** La respuesta al cliente es inmediata tras publicar en el broker.
3. **Persistencia:** El flujo termina correctamente con el almacenamiento del estado en un motor NoSQL.